

Paper Number P0329R3

Date 2017-06-06

Authors Tim Shen <timshen@google.com>
Richard Smith <richard@metafoo.co.uk>

Audience CWG

P0329R3: Designated Initialization Wording

This is a formal wording for the designated initialization proposal [P0329R0](#).

Changes compared to [P0329R2](#):

- Top-level clause number shift: 8.6.x -> 11.6.x, 13.3.x -> 16.3.x.
- Remove the use of "include" in newly added bullet at the start of 11.6.4p3.
- "the *identifiers* in the *designated-initializer-clauses*" -> "the *identifiers* in the *designators*".
- Removed "in order" in 11.6.1p3, before "where n is ...". The ordering is already described elsewhere.
- Added missing "end example" for changes in 11.6.4p3, and fixed typos.
- Changed an example for 11.6.4, demonstrating initialization with {}.
- In 11.6.1p3, exchange the two bullets after "For each explicitly initialized element".
- Remove the examples introduced in 11.6.1p3, since 11.6.1p8 already covers it.
- Remove "If there are fewer initializer-clauses ..." in 11.6.1p8, since "not explicitly initialized" is sufficient.
- Move 11.6.1p8 up.
- Rebase onto N4659.

Wording

Change 11.6 [dcl.init]p1 as follows

braced-init-list:

```
{ initializer-list ,opt }  
{ designated-initializer-list ,opt }  
{ }
```

designated-initializer-list:

designated-initializer-clause

designated-initializer-list , *designated-initializer-clause*

designated-initializer-clause:

designator brace-or-equal-initializer
designator:
. identifier

Add a new paragraph as 11.6 [dcl.init]p20:

The same *identifier* shall not appear in multiple *designators* of a *designated-initializer-list*.

Change in 11.6.4 [dcl.init.list]p1:

List-initialization is initialization of an object or reference from a *braced-init-list*. Such an initializer is called an *initializer list*, and the comma-separated *initializer-clauses* of the *initializer-list* ~~list~~ or *designated-initializer-clauses* of the *designated-initializer-list* are called the *elements* of the initializer list. [...]

Add a new bullet at the start of 11.6.4 [dcl.init.list]p3:

If the *braced-init-list* contains a *designated-initializer-list*, *T* shall be an aggregate class. The ordered *identifiers* in the *designators* of the *designated-initializer-list* shall form a subsequence of the ordered *identifiers* in the direct non-static data members of *T*. Aggregate initialization is performed ([dcl.init.aggr]). [Example:

```
struct A { int x; int y; int z; };  
A a{.y = 2, .x = 1}; // error: designator order does not match declaration order  
A b{.x = 1, .z = 2}; // ok, b.y initialized to 0
```

— end example]

Add a new paragraph to 11.6.1 [dcl.init.aggr]:

The initializations of the elements of the aggregate are evaluated in the element order. That is, all value computations and side effects associated with a given element are sequenced before those of any element that follows it in order.

Drafting note: unlike 11.6.4/4, this also covers the initialization of elements for which no initializer is explicitly provided.

Change in 11.6.1 [dcl.init.aggr]p3 and split it into two paragraphs:

When an aggregate is initialized by an initializer list as specified in 11.6.4, the elements of the initializer list are taken as initializers for the elements of the aggregate, ~~in order~~. The explicitly initialized elements of the aggregate are determined as follows:

- If the initializer list is a *designated-initializer-list*, the aggregate shall be of class type, the *identifier* in each *designator* shall name a direct non-static data member of the class, and the explicitly initialized elements of the aggregate are the elements that are, or contain, those members.
- If the initializer list is an *initializer-list*, the explicitly initialized elements of the aggregate are the first *n* elements of the aggregate, where *n* is the number of elements in the initializer list.
- Otherwise, the initializer list must be `{ }`, and there are no explicitly initialized elements.

~~Each~~ For each explicitly initialized element:

- If the element is an anonymous union object and the initializer list is a *designated-initializer-list*, the anonymous union object is initialized by the *designated-initializer-list* `{ D }`, where *D* is the *designated-initializer-clause* naming a member of the anonymous union object. There shall be only one such *designated-initializer-clause*.
- Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*. If ~~the~~ *initializer-clause* is an expression that initializer is of the form *assignment-expression* or *= assignment-expression* and a narrowing conversion (11.6.4) is required to convert the expression, the program is ill-formed. [Note: If an *initializer-clause* is itself an initializer list, the element is list-initialized, which will result in a recursive application of the rules in this section if the element is an aggregate. — end note]

[Example: ...]

Change 11.6.1 [dcl.init.aggr]p8 as follows, and move it to immediately after the above paragraphs

~~For~~ If there are fewer *initializer-clauses* in the list than there are elements in a non-union aggregate, ~~then~~ each element that is not an explicitly initialized element is initialized as follows:

...

[Example: ...]

```
struct A {
    string a;
    int b = 42;
    int c = -1;
};
```

A{.c=21} has the following steps:

1. Initialize a with {}
2. Initialize b with = 42
3. Initialize c with = 21

]

Change 11.6.1 [dcl.init.aggr]p6 as follows

[Note: Static data members, non-static data members of anonymous union members, and anonymous bit-fields are not considered elements of the aggregate. — end note]

Change 11.6.1 [dcl.init.aggr]p7 as follows

An *initializer-list* is ill-formed if the number of *initializer-clauses* exceeds the number of elements to initialize of the aggregate.

Change 11.6.1 [dcl.init.aggr]p16 as follows

When a union is initialized with an brace-enclosed initializer list, there shall not be more than one explicitly initialized element. ~~the braces shall only contain an initializer clause for the first non-static data member of the union.~~ [Example:

```
union u { int a; const char* b; };
    u a = { 1 };
    u b = a;
    u c = 1; // error
    u d = { 0, "asdf" }; // error
    u e = { "asdf" }; // error
    u f = { .b = "asdf" };
    u g = { .a = 1, .b = "asdf" }; // error
]
```

Add new paragraph after 16.3.3.1.5 [over.ics.list]p1 as follows

If the initializer list is a *designated-initializer-list*, a conversion is only possible if the parameter has an aggregate type that can be initialized from the initializer list according to the rules for aggregate initialization ([dcl.init.aggr]), in which case the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion. [Note: Aggregate initialization does not require that the members are declared in designation order. If, after overload resolution, the order does not match for the selected overload, the initialization of the parameter will be ill-formed ([dcl.init.list]). [Example:

```
struct A { int x, y; };
struct B { int y, x; };
void f(A a, int); // #1
void f(B b, ...); // #2
void g() {
    f({.x = 1, .y = 2}, 0); // OK; calls #1
    f({.y = 2, .x = 1}, 0); // error; selects #1, initialization of a fails
    // due to non-matching member order ([dcl.init.list])
}
```

____}
— end example] — end note]

Change 16.3.3.1.5 [over.ics.list]p2 as follows

Otherwise, if the parameter type is an aggregate [...]